

```
In [1]: import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: import pandas as pd
import numpy as np
import re
import emoji
import joblib
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import GridSearchCV
```

Load and Inspect Data

```
In [3]: df = pd.read_csv('Dataset/cyberbullying_tweets.csv')
```

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 47692 entries, 0 to 47691
Data columns (total 2 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_text            47692 non-null  object
1   cyberbullying_type    47692 non-null  object
dtypes: object(2)
memory usage: 745.3+ KB
```

```
In [5]: df.head()
```

```
Out[5]:
```

	tweet_text	cyberbullying_type
0	In other words #katandandre, your food was cra...	not_cyberbullying
1	Why is #aussietv so white? #MKR #theblock #ImA...	not_cyberbullying
2	@XochitlSuckkks a classy whore? Or more red ve...	not_cyberbullying
3	@Jason_Gio meh. :P thanks for the heads up, b...	not_cyberbullying
4	@RudhoeEnglish This is an ISIS account pretend...	not_cyberbullying

```
In [6]: df['cyberbullying_type'].value_counts()
```

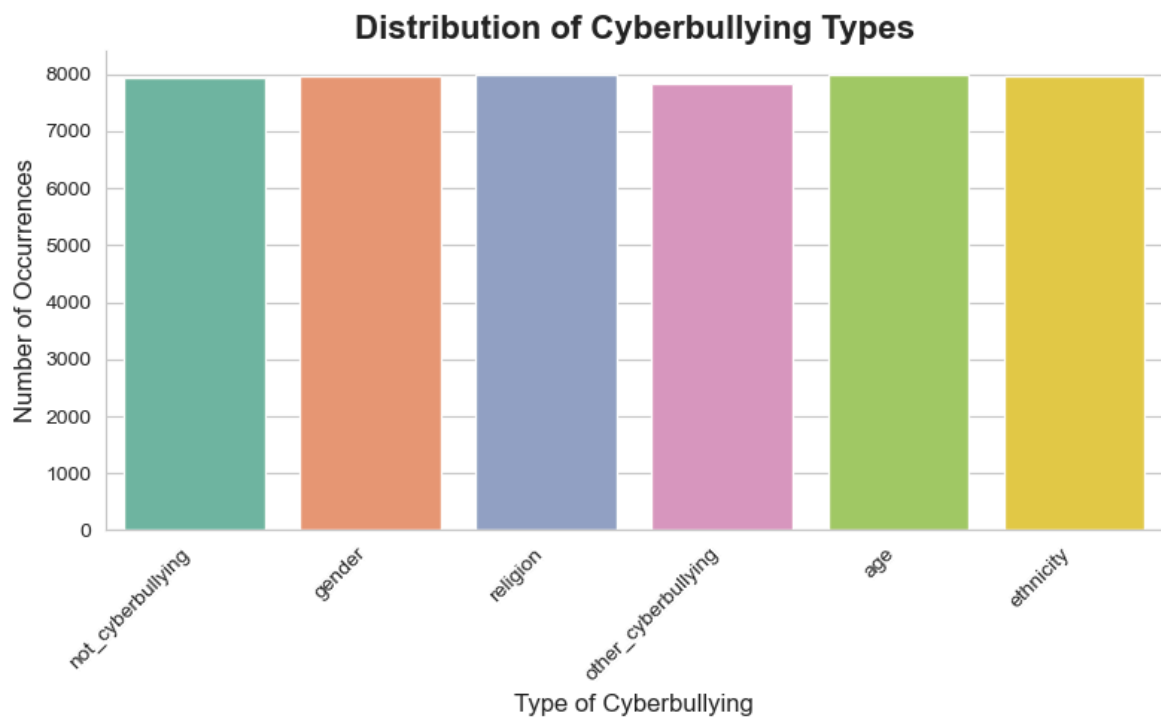
```
Out[6]: religion      7998
age      7992
gender    7973
ethnicity 7961
not_cyberbullying 7945
other_cyberbullying 7823
Name: cyberbullying_type, dtype: int64
```

Data Visualization

```
In [7]: sns.set_style("whitegrid")
plt.figure(figsize=(8, 5))
sns.countplot(x='cyberbullying_type', data=df, palette='Set2')

# Add title and Labels
plt.title('Distribution of Cyberbullying Types', fontsize=16, fontweight='bold')
plt.xlabel('Type of Cyberbullying', fontsize=12)
plt.ylabel('Number of Occurrences', fontsize=12)

# Rotate x-axis labels for better readability if needed
plt.xticks(rotation=45, ha='right')
sns.despine()
plt.tight_layout()
plt.show()
```



Preprocessing Pipeline

Lowercase Conversion

```
In [8]: df['tweet_text'] = df['tweet_text'].str.lower()
```

Emoji Translation

```
In [9]: def convert_emoji(text):
        return emoji.demojize(text, delimiters=(" ", " "))

df['tweet_text'] = df['tweet_text'].apply(convert_emoji)
```

Remove URLs and Special Characters

```
In [10]: def clean_text(text):
          text = re.sub(r"http\S+", "", text)           # Remove URLs
          text = re.sub(r"^[a-zA-Z\s]", "", text)       # Remove special characters
          return text

          df['tweet_text'] = df['tweet_text'].apply(clean_text)
```

Tokenization, Stop Words, Lemmatization

```
In [11]: stop_words = set(stopwords.words("english"))
          lemmatizer = WordNetLemmatizer()

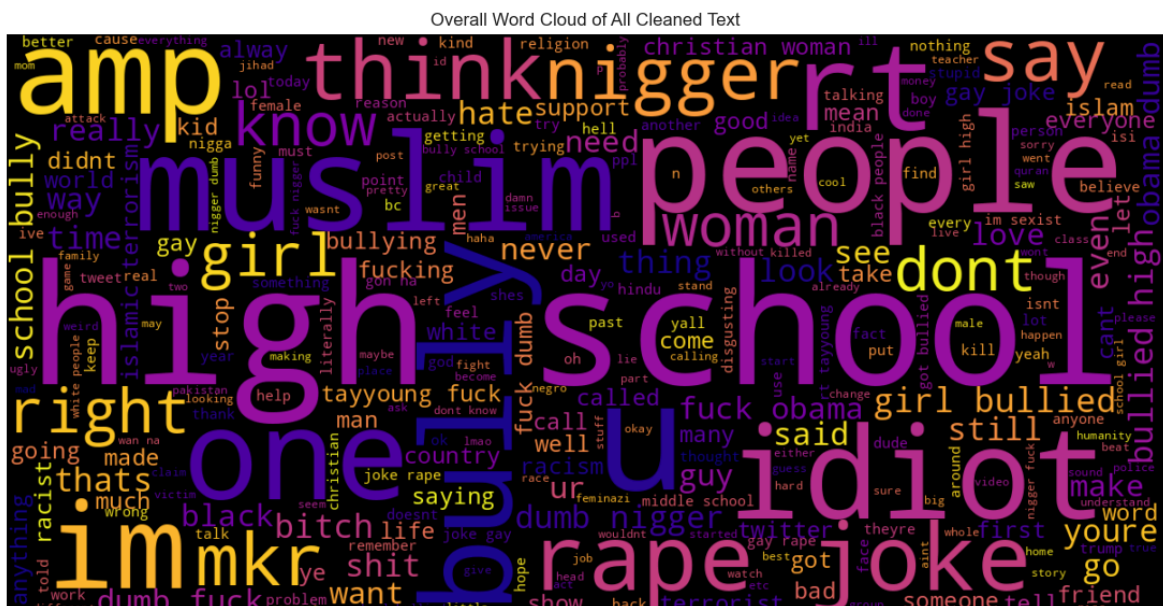
          def preprocess(text):
              tokens = word_tokenize(text)
              tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words]
              return " ".join(tokens)

          df['clean_text'] = df['tweet_text'].apply(preprocess)
```

Wordcloud

```
In [12]: all_text = " ".join(df["clean_text"])
wc = WordCloud(width=1200, height=600, background_color="Black", colormap="plasma")

plt.figure(figsize=(14,7))
plt.imshow(wc, interpolation="bilinear")
plt.axis("off")
plt.title("Overall Word Cloud of All Cleaned Text")
plt.show()
```



```
for label in labels:
    # Filter rows for this label
    text_data = df[df['cyberbullying_type'] == label]['clean_text']
    # Combine all text into one string
    combined_text = " ".join(text_data)

    # Generate word cloud
    wc = WordCloud(
        width=800,
        height=400,
        background_color='white',
        colormap='viridis',
        max_words=200
    ).generate(combined_text)

    # Plot
    plt.figure(figsize=(5,3))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    plt.title(f"Word Cloud for '{label}'")
    plt.show()
```



[illegible][illegible][illegible][illegible]

5/29


```
In [12]: tfidf = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf.fit_transform(df['clean_text'])
```

Sentiment Analysis Filtering

```
In [13]: from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

def get_sentiment(text):
    return analyzer.polarity_scores(text)

# Get sentiment scores
sentiments = df['clean_text'].apply(get_sentiment)

# Convert to DataFrame and reset index to match df
df_sentiments = pd.DataFrame(sentiments.tolist()).reset_index(drop=True)
df = df.reset_index(drop=True)

negative_mask = df_sentiments['compound'] < 0
df = df[negative_mask]
X_tfidf = X_tfidf[negative_mask.values]
```

Label Encoding

```
In [14]: from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y = le.fit_transform(df['cyberbullying_type'])
```

```
In [15]: X = X_tfidf
```

```
In [16]: import os
import joblib

os.makedirs("Models", exist_ok=True)

joblib.dump(tfidf, "Models/tfidf_vectorizer.pkl")
joblib.dump(le, "Models/label_encoder.pkl")
```

```
Out[16]: ['Models/label_encoder.pkl']
```

Oversampling with SMOTE

```
In [17]: from imblearn.over_sampling import SMOTE
smt = SMOTE(k_neighbors=2)
X_res, y_res = smt.fit_resample(X, y)
```

Train-Test Split

```
In [18]: X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=0.20)
```

```
In [19]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMa
```

```
ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []

#function to call for storing the results
def storeResults(model, a,b,c,d):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(d, 3))
    recall.append(round(c, 3))
    f1score.append(round(b, 3))
```

LogisticRegression

```
In [22]: from sklearn.linear_model import LogisticRegression

model_path = "Models/lr_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        lr_best = joblib.load(f)
else:
    param_grid = {
        "C": [0.1, 1, 10],
        "solver": ["lbfgs", "saga"]
    }

    grid = GridSearchCV(
        LogisticRegression(
            multi_class="multinomial",
            max_iter=1000
        ),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    lr_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(lr_best, f)

y_pred = lr_best.predict(X_test)

lr_acc = accuracy_score(y_test, y_pred)
lr_prec = precision_score(y_test, y_pred, average="weighted")
lr_rec = recall_score(y_test, y_pred, average="weighted")
```

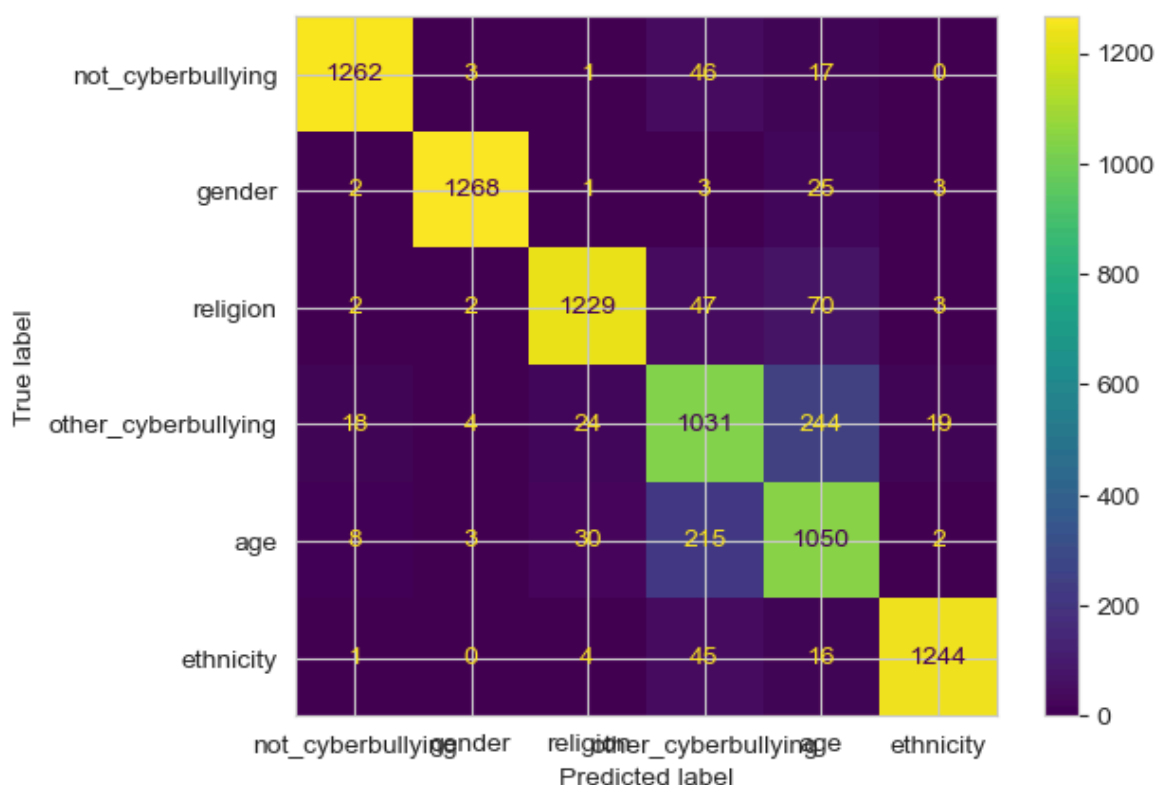
```
lr_f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy      : {lr_acc:.4f}")
print(f"Precision     : {lr_prec:.4f}")
print(f"Recall        : {lr_rec:.4f}")
print(f"F1 Score      : {lr_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")
```

```
Accuracy      : 0.8920
Precision     : 0.8966
Recall        : 0.8920
F1 Score      : 0.8939
```

Out[22]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x242714b5ea0>



In [23]: storeResults('LogisticRegression', lr_acc, lr_prec, lr_rec, lr_f1)

Random Forest

```
In [24]: from sklearn.ensemble import RandomForestClassifier

model_path = "Models/rf_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        rf_best = joblib.load(f)
else:
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [None, 10, 20],
        "max_features": ["sqrt", "log2"]
```



```

    }

    grid = GridSearchCV(
        RandomForestClassifier(random_state=0),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    rf_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(rf_best, f)

y_pred = rf_best.predict(X_test)

rf_acc = accuracy_score(y_test, y_pred)
rf_prec = precision_score(y_test, y_pred, average="weighted")
rf_rec = recall_score(y_test, y_pred, average="weighted")
rf_f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy          : {rf_acc:.4f}")
print(f"Precision         : {rf_prec:.4f}")
print(f"Recall             : {rf_rec:.4f}")
print(f"F1 Score           : {rf_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

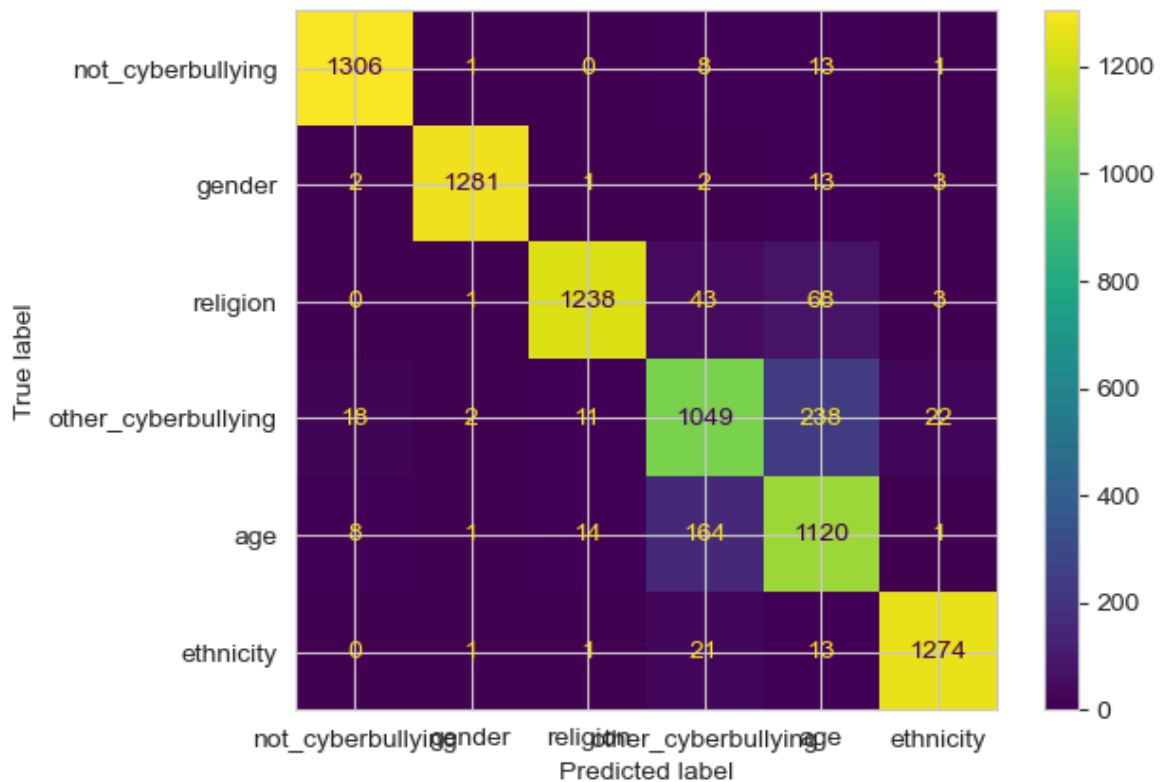
Accuracy          : 0.9151
Precision         : 0.9183
Recall           : 0.9151
F1 Score         : 0.9160

```

```

Out[24]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2426b912800>

```



```
In [25]: storeResults('RandomForest', rf_acc, rf_prec, rf_rec, rf_f1)
```

XGBoost

```
In [26]: from xgboost import XGBClassifier

model_path = "Models/xgb_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        xgb_best = joblib.load(f)
else:
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [3, 5, 7],
        "learning_rate": [0.05, 0.1, 0.3]
    }

    grid = GridSearchCV(
        XGBClassifier(use_label_encoder=False, eval_metric="mlogloss"),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    xgb_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(xgb_best, f)
```

```

y_pred = xgb_best.predict(X_test)

xgb_acc = accuracy_score(y_test, y_pred)
xgb_prec = precision_score(y_test, y_pred, average="weighted")
xgb_rec = recall_score(y_test, y_pred, average="weighted")
xgb_f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy      : {xgb_acc:.4f}")
print(f"Precision     : {xgb_prec:.4f}")
print(f"Recall        : {xgb_rec:.4f}")
print(f"F1 Score      : {xgb_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

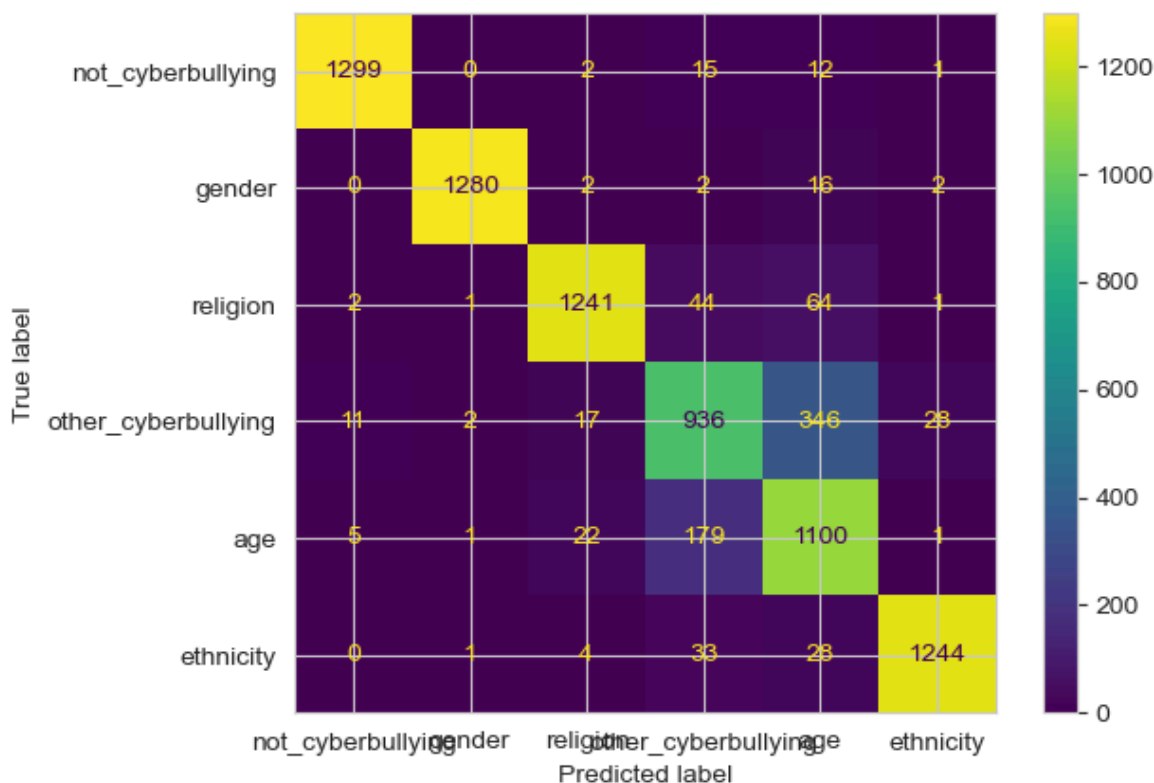
Accuracy      : 0.8940
Precision     : 0.8995
Recall        : 0.8940
F1 Score      : 0.8952

```

```

Out[26]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x24271417a00>

```



```

In [27]: storeResults("XGBoost", xgb_acc, xgb_prec, xgb_rec, xgb_f1)

```

DecisionTree

```

In [28]: from sklearn.tree import DecisionTreeClassifier

model_path = "Models/dt_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        dt_best = joblib.load(f)

```

```

else:
    param_grid = {
        "max_depth": [None, 5, 10, 20],
        "criterion": ["gini", "entropy"]
    }

    grid = GridSearchCV(
        DecisionTreeClassifier(random_state=0),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    dt_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(dt_best, f)

y_pred = dt_best.predict(X_test)

dt_acc = accuracy_score(y_test, y_pred)
dt_prec = precision_score(y_test, y_pred, average="weighted")
dt_rec = recall_score(y_test, y_pred, average="weighted")
dt_f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy          : {dt_acc:.4f}")
print(f"Precision         : {dt_prec:.4f}")
print(f"Recall             : {dt_rec:.4f}")
print(f"F1 Score           : {dt_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

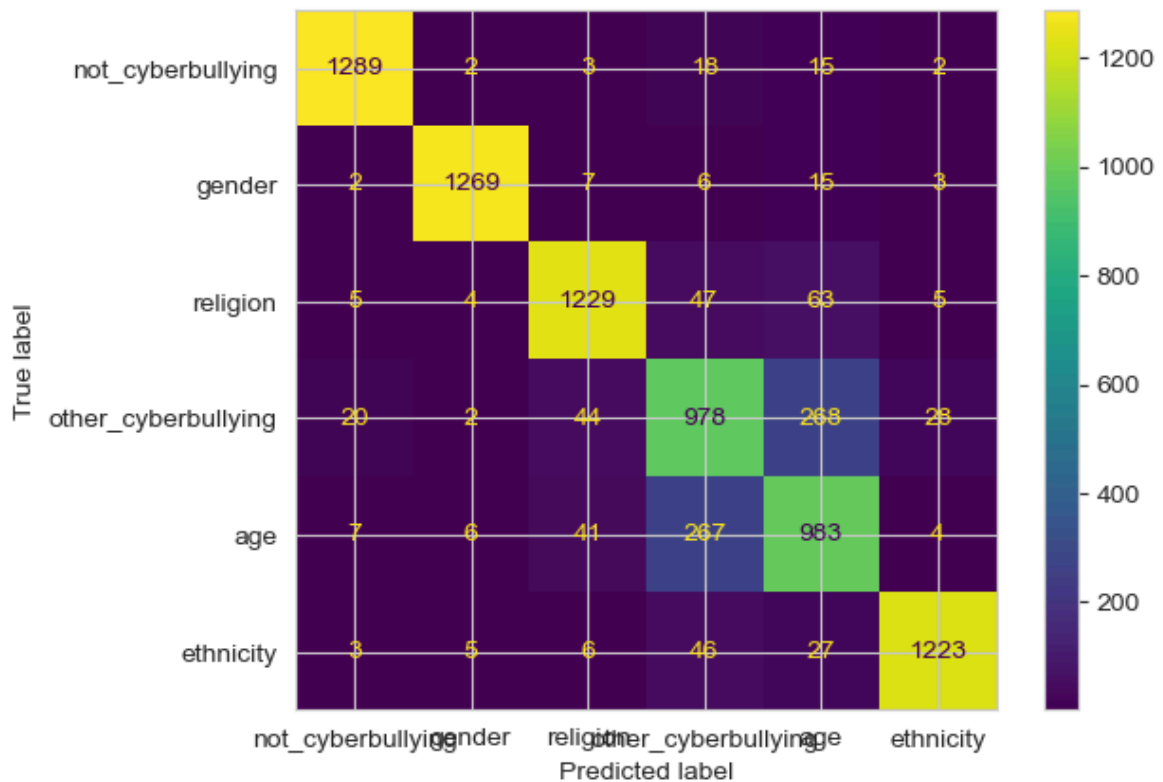
Accuracy          : 0.8777
Precision         : 0.8803
Recall           : 0.8777
F1 Score         : 0.8789

```

```

Out[28]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x242677776d0>

```



```
In [29]: storeResults("DecisionTree", dt_acc, dt_prec, dt_rec, dt_f1)
```

Naive Bayes

```
In [30]: from sklearn.naive_bayes import MultinomialNB
```

```
model_path = "Models/nb_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        nb_best = joblib.load(f)
else:
    param_grid = {
        "alpha": [0.1, 0.5, 1.0]
    }

    grid = GridSearchCV(
        MultinomialNB(),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    nb_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(nb_best, f)

y_pred = nb_best.predict(X_test)
```

```

nb_acc = accuracy_score(y_test, y_pred)
nb_prec = precision_score(y_test, y_pred, average="weighted")
nb_rec = recall_score(y_test, y_pred, average="weighted")
nb_f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy      : {nb_acc:.4f}")
print(f"Precision     : {nb_prec:.4f}")
print(f"Recall         : {nb_rec:.4f}")
print(f"F1 Score       : {nb_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

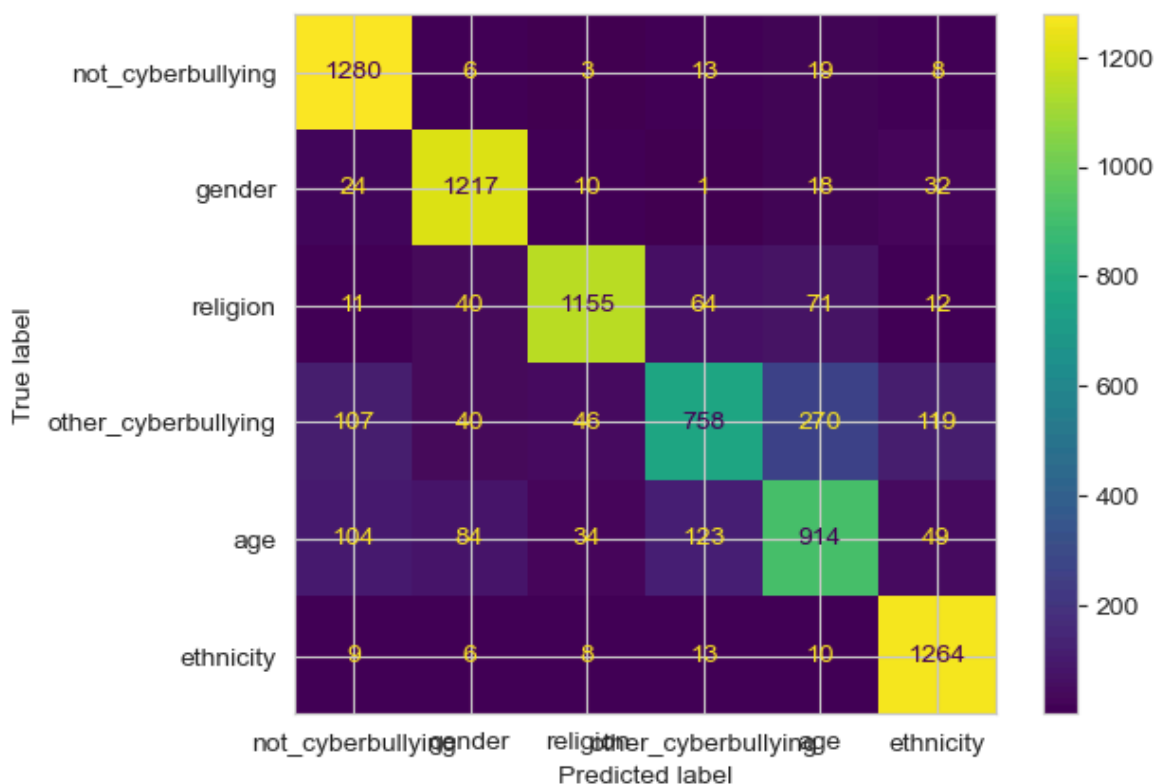
Accuracy      : 0.8295
Precision     : 0.8271
Recall        : 0.8295
F1 Score      : 0.8237

```

```

Out[30]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x242724f5a20>

```



```

In [31]: storeResults('NaiveBayes', nb_acc, nb_prec, nb_rec, nb_f1)

```

SVM

```

In [32]: from sklearn.svm import SVC

model_path = "Models/svm_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        svm_best = joblib.load(f)
else:

```



```

param_grid = {
    "C": [0.1, 1, 10],
    "kernel": ["linear", "rbf"],
    "gamma": ["scale", "auto"]
}

grid = GridSearchCV(
    SVC(probability=True),
    param_grid=param_grid,
    scoring="f1_weighted",
    cv=3,
    n_jobs=-1,
    verbose=1
)

grid.fit(X_train, y_train)
svm_best = grid.best_estimator_

# Save model
with open(model_path, "wb") as f:
    joblib.dump(svm_best, f)

y_pred = svm_best.predict(X_test)

svm_acc = accuracy_score(y_test, y_pred)
svm_prec = precision_score(y_test, y_pred, average='weighted')
svm_rec = recall_score(y_test, y_pred, average='weighted')
svm_f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy      : {svm_acc:.4f}")
print(f"Precision     : {svm_prec:.4f}")
print(f"Recall         : {svm_rec:.4f}")
print(f"F1 Score       : {svm_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

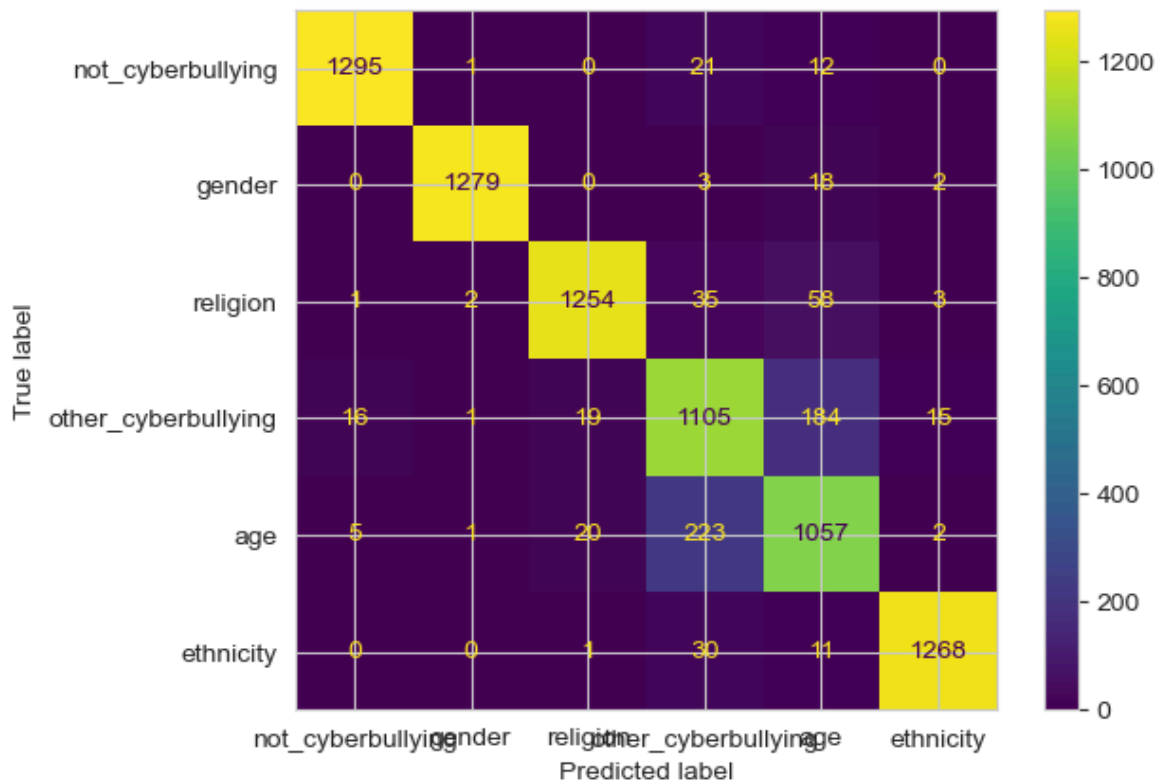
Accuracy      : 0.9139
Precision     : 0.9166
Recall        : 0.9139
F1 Score      : 0.9150

```

```

Out[32]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x24274d1cd90>

```



```
In [33]: storeResults('SVM', svm_acc, svm_prec, svm_rec, svm_f1)
```

ExtraTrees

```
In [34]: from sklearn.ensemble import ExtraTreesClassifier
```

```
model_path = "Models/extree_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        extree_best = joblib.load(f)
else:
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [None, 10, 20],
        "max_features": ["sqrt", "log2"]
    }

    grid = GridSearchCV(
        ExtraTreesClassifier(random_state=0),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    extree_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(extree_best, f)
```

```

y_pred = extree_best.predict(X_test)

extree_acc = accuracy_score(y_test, y_pred)
extree_prec = precision_score(y_test, y_pred, average='weighted')
extree_rec = recall_score(y_test, y_pred, average='weighted')
extree_f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy      : {extree_acc:.4f}")
print(f"Precision     : {extree_prec:.4f}")
print(f"Recall          : {extree_rec:.4f}")
print(f"F1 Score        : {extree_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

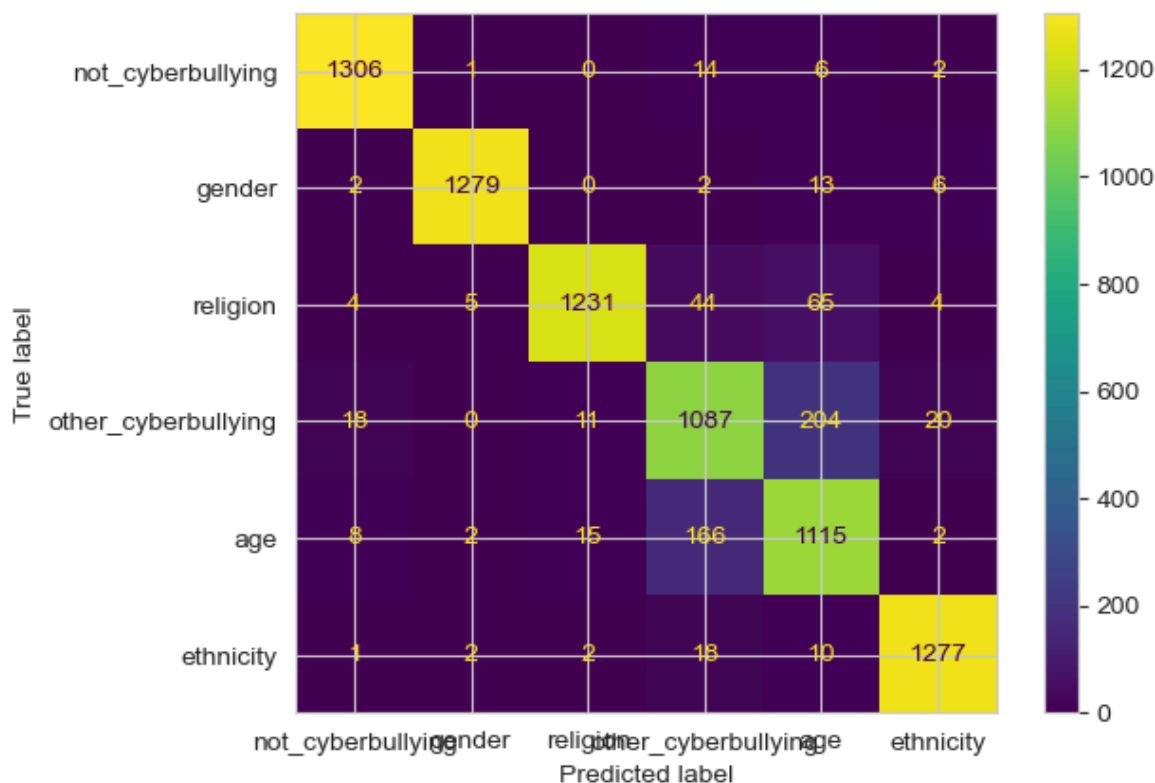
Accuracy      : 0.9185
Precision     : 0.9209
Recall        : 0.9185
F1 Score      : 0.9193

```

```

Out[34]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2427040beb0>

```



```

In [35]: storeResults('ExtraTrees', extree_acc, extree_prec, extree_rec, extree_f1)

```

GradientBoosting

```

In [36]: from sklearn.ensemble import GradientBoostingClassifier

model_path = "Models/gb_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        gb_best = joblib.load(f)

```

```

else:
    param_grid = {
        "n_estimators": [100, 200],
        "learning_rate": [0.05, 0.1, 0.5, 1.0],
        "max_depth": [1, 3, 5]
    }

    grid = GridSearchCV(
        GradientBoostingClassifier(random_state=0),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    gb_best = grid.best_estimator_

    with open(model_path, "wb") as f:
        joblib.dump(gb_best, f)

y_pred = gb_best.predict(X_test)

gb_acc = accuracy_score(y_test, y_pred)
gb_prec = precision_score(y_test, y_pred, average='weighted')
gb_rec = recall_score(y_test, y_pred, average='weighted')
gb_f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy          : {gb_acc:.4f}")
print(f"Precision         : {gb_prec:.4f}")
print(f"Recall              : {gb_rec:.4f}")
print(f"F1 Score            : {gb_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap='viridis')

```

```

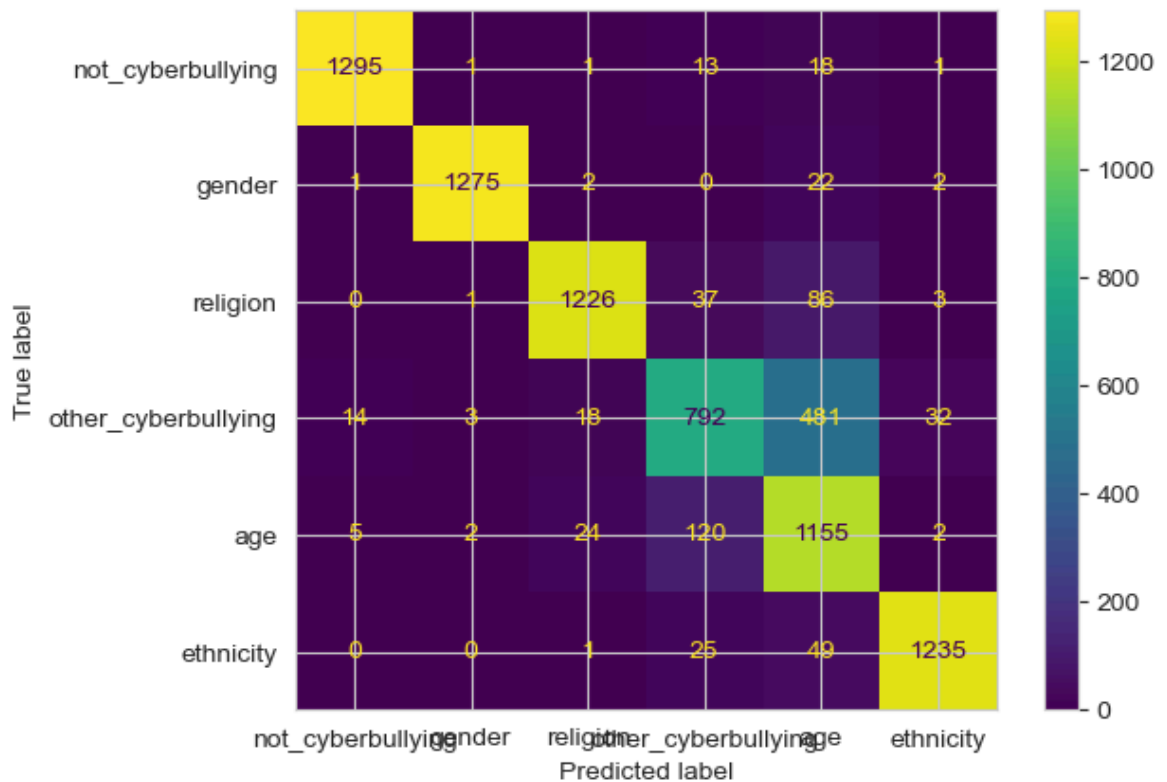
Accuracy          : 0.8786
Precision         : 0.8922
Recall            : 0.8786
F1 Score          : 0.8793

```

```

Out[36]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x24270393c40>

```



```
In [37]: storeResults('GradientBoosting', gb_acc, gb_prec, gb_rec, gb_f1)
```

AdaBoost

```
In [38]: from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

model_path = "Models/ada_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        ab_best = joblib.load(f)
else:
    base_est = DecisionTreeClassifier()

    param_grid = {
        "n_estimators": [50, 100],
        "learning_rate": [0.5, 1.0],
        "base_estimator__max_depth": [1, 2]
    }

    grid = GridSearchCV(
        AdaBoostClassifier(base_estimator=base_est),
        param_grid=param_grid,
        scoring="f1_weighted",
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    ab_best = grid.best_estimator_
```

```

with open(model_path, "wb") as f:
    joblib.dump(ab_best, f)

y_pred = ab_best.predict(X_test)

ab_acc = accuracy_score(y_test, y_pred)
ab_prec = precision_score(y_test, y_pred, average='weighted')
ab_rec = recall_score(y_test, y_pred, average='weighted')
ab_f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy      : {ab_acc:.4f}")
print(f"Precision     : {ab_prec:.4f}")
print(f"Recall         : {ab_rec:.4f}")
print(f"F1 Score      : {ab_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="viridis")

```

```

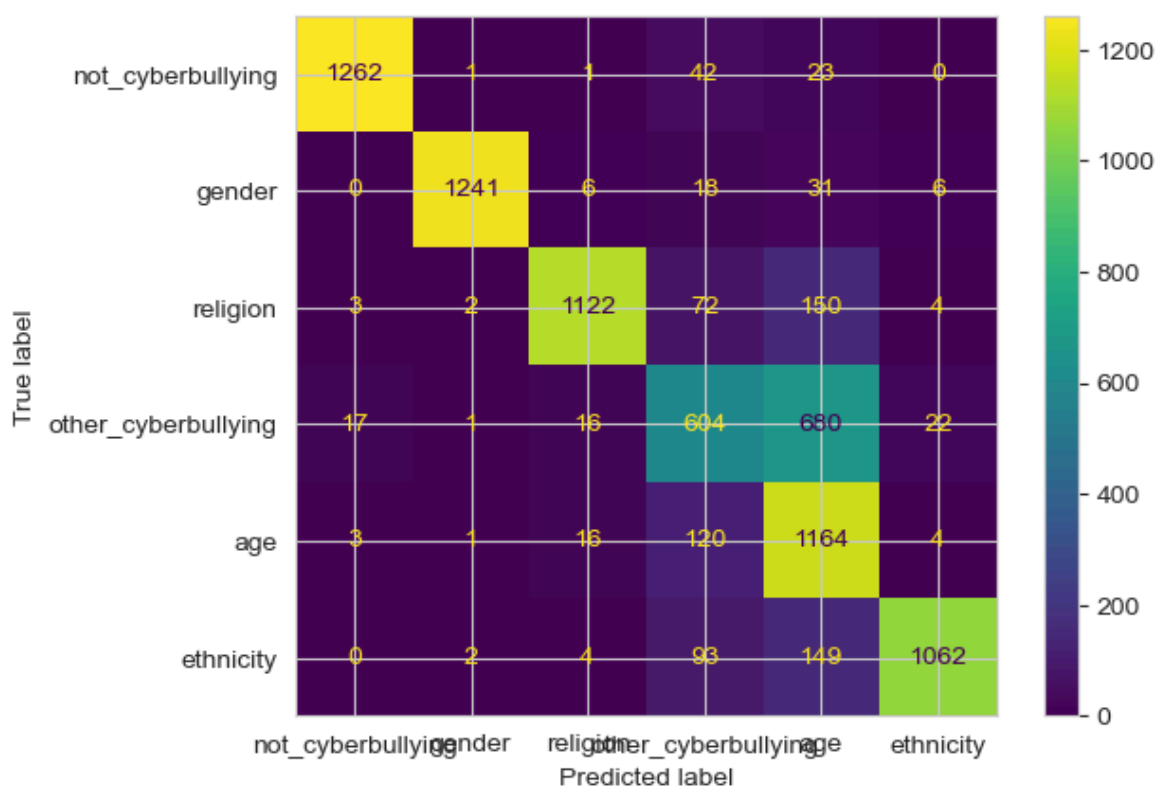
Accuracy      : 0.8128
Precision     : 0.8456
Recall        : 0.8128
F1 Score      : 0.8169

```

```

Out[38]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x242704551b0>

```



```

In [39]: storeResults('AdaBoost', ab_acc, ab_prec, ab_rec, ab_f1)

```

Extension

Oversampling with SMOTEENN

```
In [20]: from imblearn.combine import SMOTEENN

smt = SMOTEENN(random_state=42)
X_res, y_res = smt.fit_resample(X, y)
```

Train-Test Split

```
In [21]: X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=0.20)
```

Voting Classifier

```
In [24]: from sklearn.ensemble import VotingClassifier, BaggingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression

mlp = MLPClassifier(random_state=10, max_iter=1000)
brf = BaggingClassifier(
    base_estimator=LogisticRegression(random_state=0),
    n_estimators=10,
    random_state=0,
    max_samples=1.0,
    max_features=1.0
)

vot = VotingClassifier(
    estimators=[('MLP', mlp), ('BagRF', brf)],
    voting='soft'
)

model_path = "Models/voting_model.pkl"

if os.path.exists(model_path):
    with open(model_path, "rb") as f:
        vot = joblib.load(f)
else:
    param_grid = {
        'MLP__hidden_layer_sizes': [(100,), (50,50)],
        'MLP__activation': ['relu', 'tanh'],
        'MLP__alpha': [0.0001, 0.001],
        'BagRF__n_estimators': [10, 20],
        'BagRF__base_estimator__C': [0.1, 1, 10]
    }

    grid = GridSearchCV(
        estimator=vot,
        param_grid=param_grid,
        scoring='f1_weighted',
        cv=3,
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    vot = grid.best_estimator_
```

```

# Save model
os.makedirs("Models", exist_ok=True)
with open(model_path, "wb") as f:
    joblib.dump(vot, f)

y_pred = vot.predict(X_test)

vot_acc = accuracy_score(y_test, y_pred)
vot_prec = precision_score(y_test, y_pred, average='weighted')
vot_rec = recall_score(y_test, y_pred, average='weighted')
vot_f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy      : {vot_acc:.4f}")
print(f"Precision     : {vot_prec:.4f}")
print(f"Recall        : {vot_rec:.4f}")
print(f"F1 Score      : {vot_f1:.4f}")

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap='viridis')

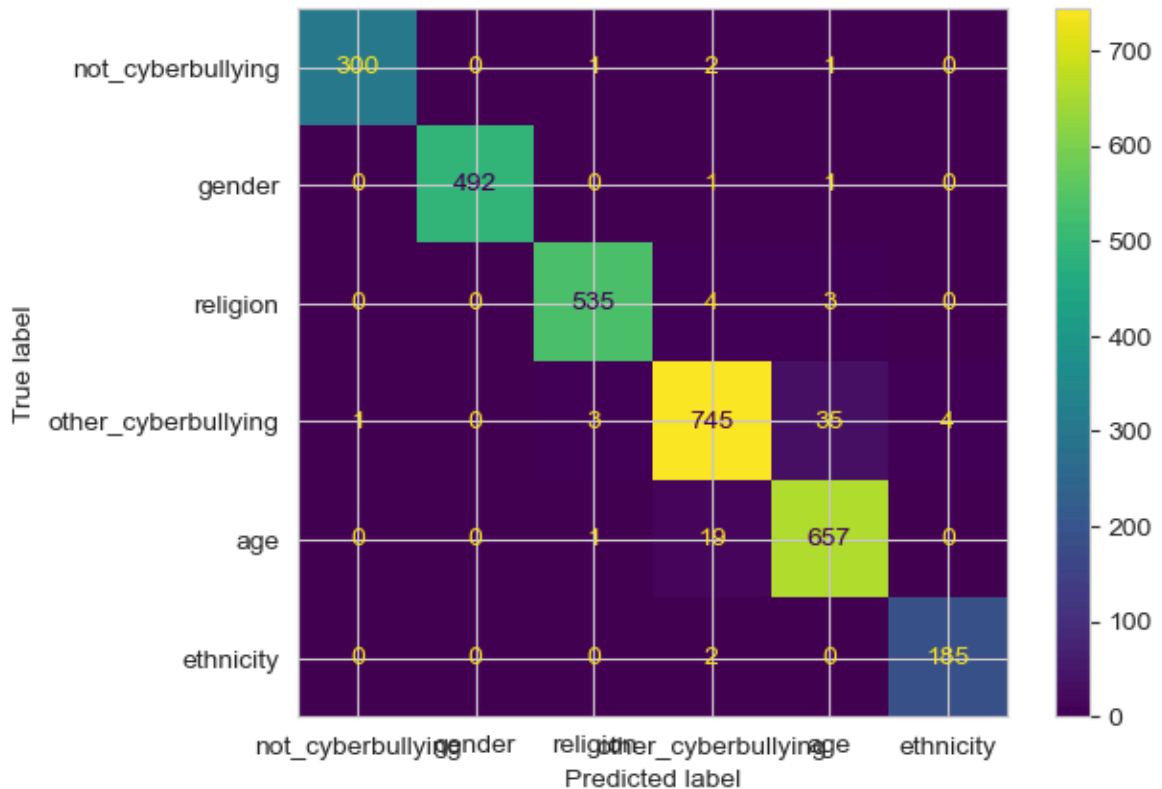
```

```

Accuracy      : 0.9739
Precision     : 0.9741
Recall       : 0.9739
F1 Score     : 0.9740

```

Out[24]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b0c0c6ad40>



In [25]: storeResults('Voting Classifier', vot_acc, vot_prec, vot_rec, vot_f1)

Comparison


```
In [44]: #creating dataframe
result = pd.DataFrame({ 'ML Model' : ML_Model,
                        'Accuracy' : accuracy,
                        'Precision': precision,
                        'Recall'   : recall,
                        'F1-Score' : f1score,
                        })
```

```
In [45]: result
```

```
Out[45]:
```

	ML Model	Accuracy	Precision	Recall	F1-Score
0	LogisticRegression	0.892	0.894	0.892	0.897
1	RandomForest	0.915	0.916	0.915	0.918
2	XGBoost	0.894	0.895	0.894	0.899
3	DecisionTree	0.878	0.879	0.878	0.880
4	NaiveBayes	0.830	0.824	0.830	0.827
5	SVM	0.914	0.915	0.914	0.917
6	ExtraTrees	0.919	0.919	0.919	0.921
7	GradientBoosting	0.879	0.879	0.879	0.892
8	AdaBoost	0.813	0.817	0.813	0.846
9	Voting Classifier	0.974	0.974	0.974	0.974

Graphs

```
In [26]: from lime.lime_text import LimeTextExplainer

class_names = list(set(y))
lime_explainer = LimeTextExplainer(class_names=class_names)
```

```
In [27]: def predict_proba_lime(texts):
          return vot.predict_proba(tfidf.transform(texts))
```

```
In [28]: print("\n--- LIME ---")
          for i in range(3):
              idx = i
              sample_text = df['clean_text'].iloc[idx]
              exp = lime_explainer.explain_instance(sample_text, predict_proba_lime, num_f
              print(f"\nSample {idx}: {sample_text}")
              print(exp.as_list())
              exp.show_in_notebook(text=True)
```

--- LIME ---

Sample 0: xochitlsuckkks classy whore red velvet cupcake
 [('whore', 0.0005410846208354837), ('red', -0.0002469300829633257), ('classy', -0.0001468506874660791), ('cupcake', 2.4698960941369452e-05), ('xochitlsuckkks', 1.7561321947996782e-05), ('velvet', 7.301174907420885e-06)]

Prediction probabilities



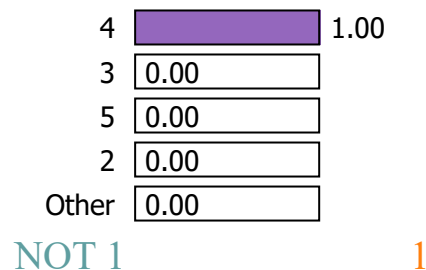
whore
0.00
red
0.00
classy
0.00
cupcake
0.00
xochitlsuckkks
0.00
velvet
0.00

Text with highlighted words

xochitlsuckkks classy whore red velvet cupcake

Sample 1: jasongio meh p thanks head concerned another angry dude twitter
 [('angry', -0.000907189564306221), ('twitter', -0.00042185218643226265), ('another', 0.00025118865463600356), ('head', 0.00023601981405679478), ('concerned', -0.0022347377615311904), ('dude', -0.00021292997705913138), ('meh', -6.20654076678289e-05), ('thanks', -2.6351427404597475e-05)]

Prediction probabilities



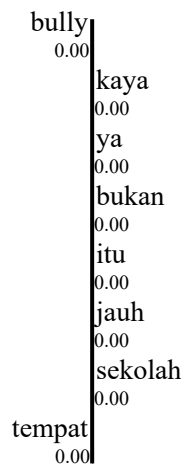
angry
0.00
twitter
0.00
another
0.00
head
0.00
concerned
0.00
dude
0.00
meh
0.00
thanks
0.00

Text with highlighted words

jasongio meh p thanks head concerned another angry dude twitter

Sample 2: itu sekolah ya bukan tempat bully ga jauh kaya neraka
 [('bully', -0.0019719239993684205), ('kaya', 0.0009111788864936888), ('ya', 8.07187713371728e-05), ('bukan', 1.0463753289381452e-05), ('itu', 8.873117957065115e-06), ('jauh', 6.972226217177252e-06), ('sekolah', 6.130506319111559e-06), ('tempat', -6.013450795917058e-06)]

Prediction probabilities



Text with highlighted words

itu sekolah ya bukan tempat **bully** ga jauh **kaya** neraka

```
In [29]: import shap
```

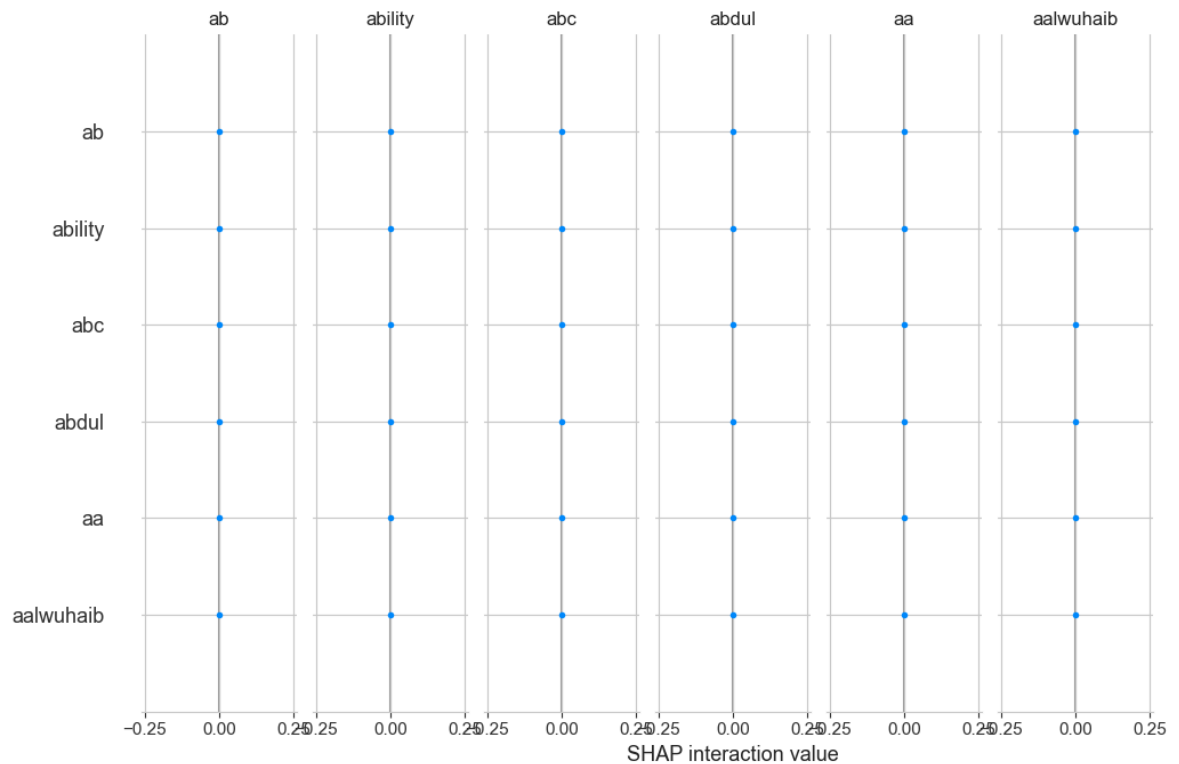
```
In [32]: print("\n--- SHAP EXPLANATION ---")
X_sample = X_test[:20].toarray()

explainer = shap.KernelExplainer(vot.predict_proba, X_sample[:5])

shap_values = explainer.shap_values(X_sample[0:1])
```

```
--- SHAP EXPLANATION ---
0%|          | 0/1 [00:00<?, ?it/s]
```

```
In [34]: shap.summary_plot(shap_values, feature_names=tfidf.get_feature_names_out())
```



```
In [36]: exp = shap.Explanation(values=shap_values[0][0],
                                base_values=explainer.expected_value[0],
                                data=X_sample[0],
                                feature_names=tfidf.get_feature_names_out())
```

```
In [37]: # Waterfall plot
shap.plots.waterfall(exp)
```

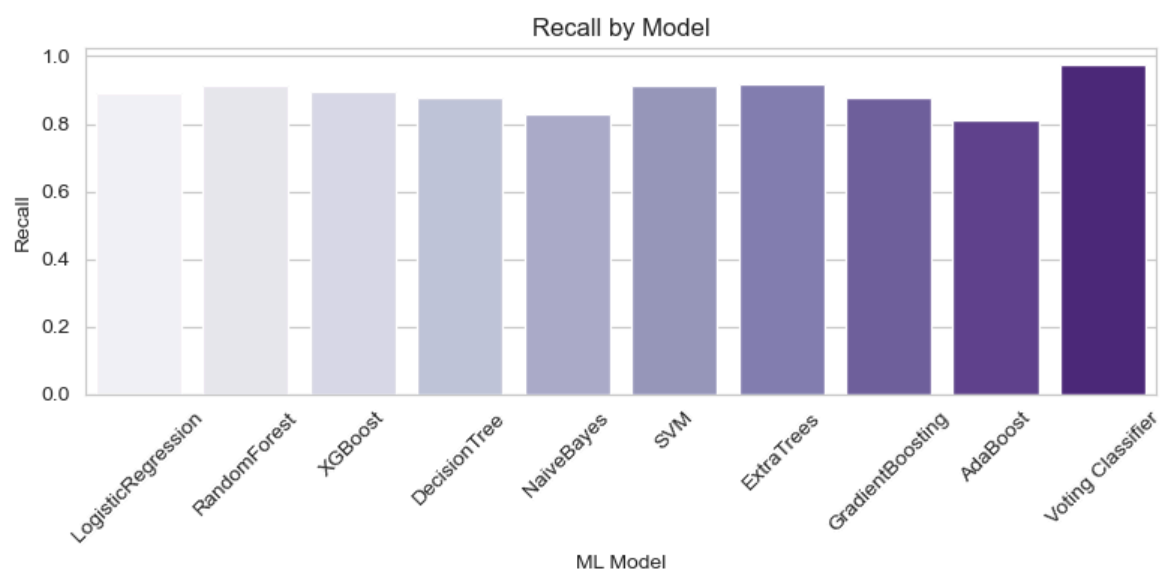
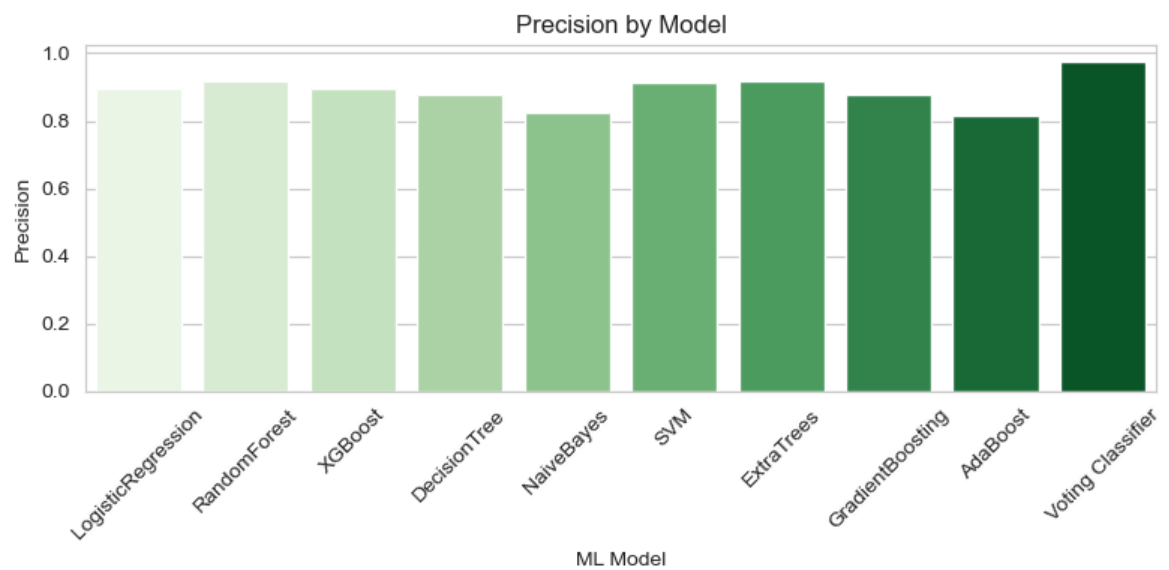
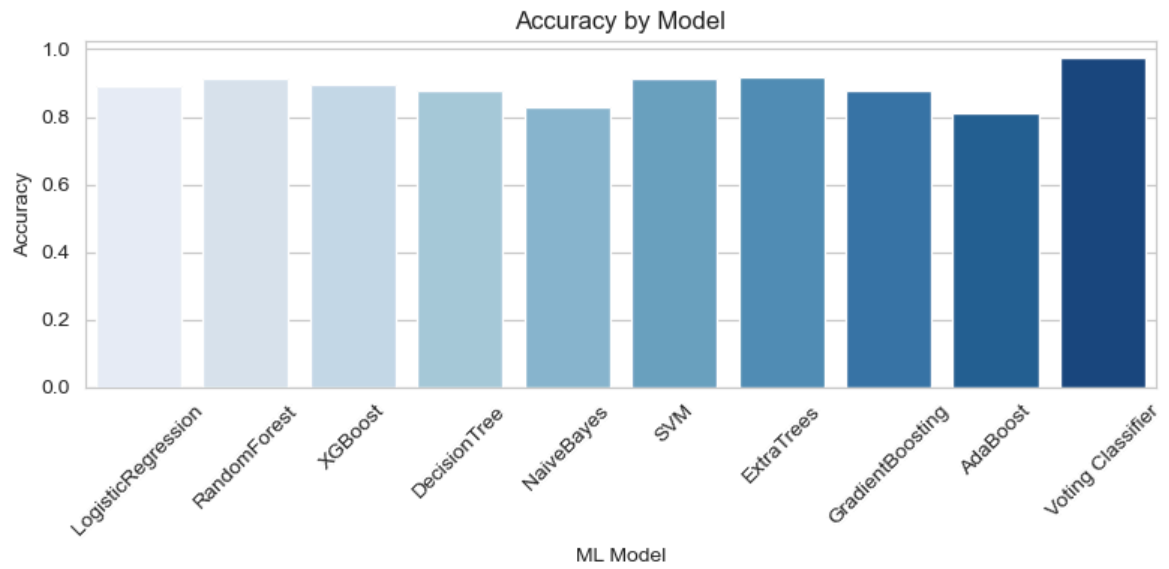


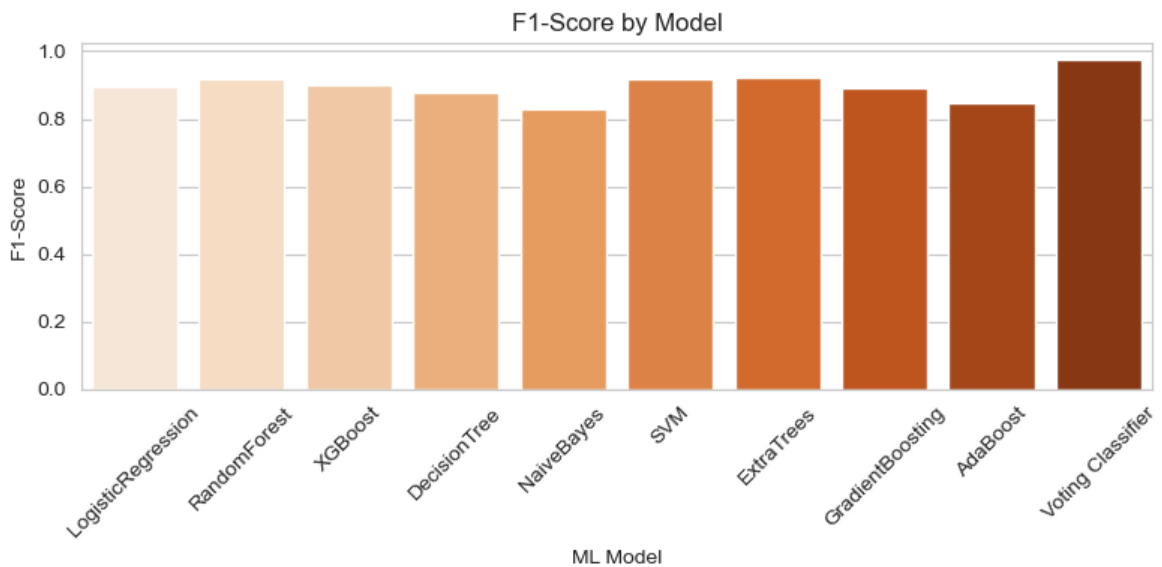
```
In [ ]:
```

```
In [46]: import seaborn as sns
import matplotlib.pyplot as plt

# Define a different palette for each metric
palettes = {
    'Accuracy': 'Blues',
    'Precision': 'Greens',
    'Recall': 'Purples',
    'F1-Score': 'Oranges'
}

for metric in ['Accuracy', 'Precision', 'Recall', 'F1-Score']:
    plt.figure(figsize=(8, 4))
    sns.barplot(data=result, x='ML Model', y=metric, palette=palettes[metric])
    plt.title(f'{metric} by Model')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```





Prediction

```
In [47]: import joblib
import re
import emoji
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

# Load saved components
model = joblib.load("Models/voting_model.pkl")
tfidf = joblib.load("Models/tfidf_vectorizer.pkl")

# Preprocessing tools
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
sentiment_analyzer = SentimentIntensityAnalyzer()

# Preprocessing function
def preprocess_text(text):
    text = text.lower()
    text = emoji.demojize(text, delimiters=(" ", " "))
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"^[a-zA-Z\s]", "", text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words]
    return " ".join(tokens)
```

```
In [48]: new_text = "You are such an idiot and nobody likes you!"
```

```
In [49]: message = preprocess_text(new_text)
data = [message]

# Vectorize
vect = tfidf.transform(data).toarray()

# Predict
result = model.predict(vect)[0]
```

```
# Output  
print(f"Predicted Class Index: {result}")
```

Predicted Class Index: 4

```
In [50]: new_text = "In other words #katandandre, your food was crapilicious! #mkr"
```

```
In [51]: message = preprocess_text(new_text)  
data = [message]  
  
# Vectorize  
vect = tfidf.transform(data).toarray()  
  
# Predict  
result = model.predict(vect)[0]  
  
# Output  
print(f"Predicted Class Index: {result}")
```

Predicted Class Index: 3

```
In [52]: new_text = "Here at home. Neighbors pick on my family and I. Mind you my son is
```

```
In [53]: message = preprocess_text(new_text)  
data = [message]  
  
# Vectorize  
vect = tfidf.transform(data).toarray()  
  
# Predict  
result = model.predict(vect)[0]  
  
# Output  
print(f"Predicted Class Index: {result}")
```

Predicted Class Index: 0

```
In [ ]:
```